

Selenium Automation Framework — Project Report

Project Name: selenium.automation.framework

Target Application: Demo Web Shop (<https://demowebshop.tricentis.com>)

Tech Stack: Java 11 | Selenium 4.18.1 | TestNG 7.9.0 | Maven 3.9 | ExtentReports 5.1

1. Project Overview

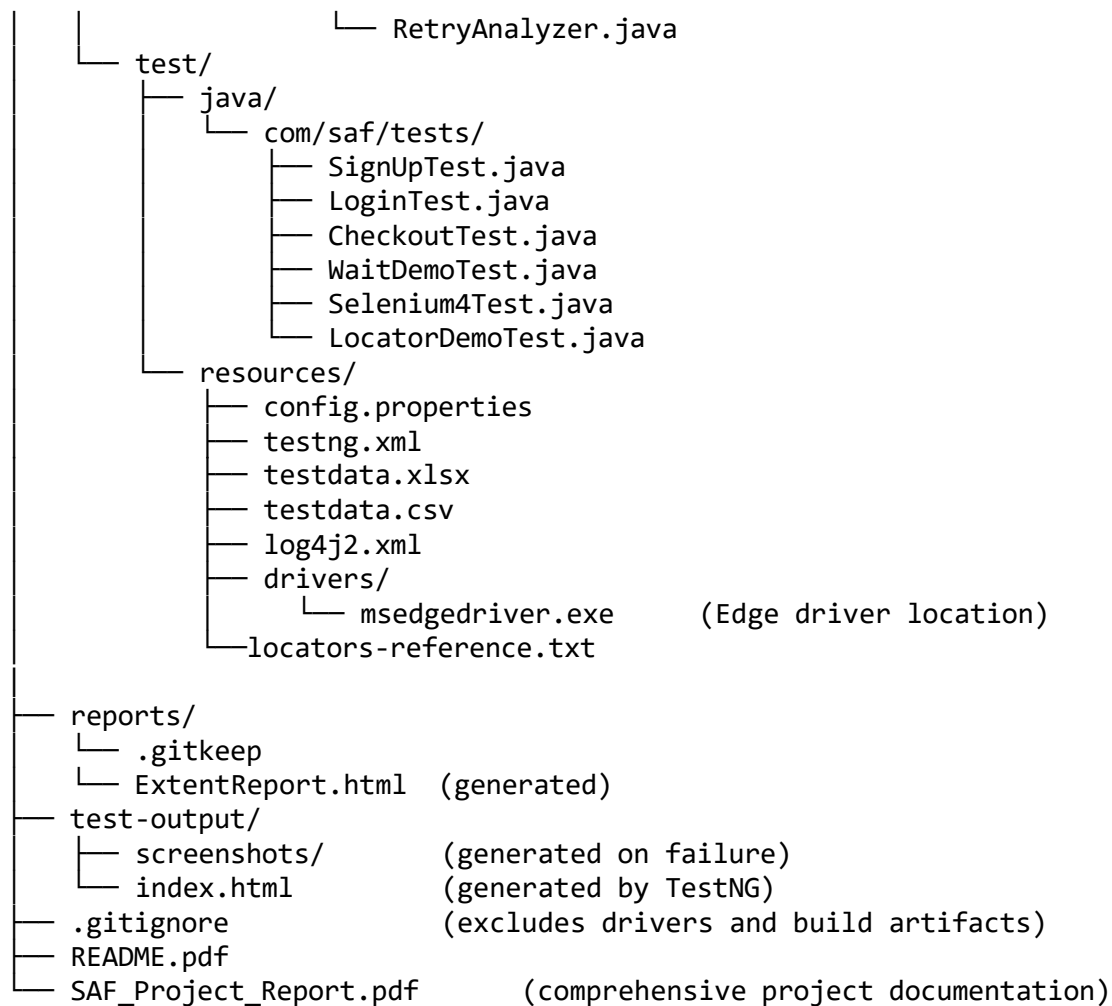
This Selenium Automation Framework demonstrates comprehensive test automation capabilities for a modern e-commerce platform. The project was built to solve real-world challenges in web application testing, including dynamic content handling, multi-browser compatibility, and robust synchronization strategies. By automating the complete user journey on Demo Web Shop—from registration through checkout—this framework showcases practical application of Selenium WebDriver concepts in a production-like environment.

The framework addresses common automation pain points such as element synchronization, test data management, and cross-browser execution. It provides a solid foundation for enterprise-level test automation with features like parallel execution, comprehensive reporting, and data-driven testing capabilities. The switch from originally planned OpenCart site to Demo Web Shop was necessitated by Cloudflare protection on the former, but ultimately strengthened the project by utilizing a more stable and automation-friendly e-commerce platform. The site change was confirmed and approved by PCG support provided by Sachin Tiwari from Internshala.

2. Framework Architecture

2.1 Directory Structure

```
Selenium_Automation_Framework /
├── pom.xml
├── src/
│   ├── main/
│   │   └── java/
│   │       ├── com/saf/
│   │       │   ├── base/
│   │       │   │   └── BaseTest.java
│   │       │   ├── listeners/
│   │       │   │   └── TestListener.java
│   │       │   └── utils/
│   │       │       ├── ConfigReader.java
│   │       │       ├── ExcelUtil.java
│   │       │       ├── WaitUtil.java
│   │       │       └── ScreenshotUtil.java
```



2.2 Architectural Decisions

The framework follows a layered architecture with clear separation of concerns. BaseTest serves as the parent class providing browser initialization, configuration management, and lifecycle hooks through TestNG annotations. Utility classes reside in the main/java package rather than test/java to promote reusability across different test suites and potential future integration with other frameworks. The TestListener is isolated to handle cross-cutting concerns like reporting and screenshot capture, following the single responsibility principle.

The ThreadLocal pattern is implemented to ensure thread safety during parallel execution, preventing driver interference between concurrent test threads. This design enables the framework to scale horizontally across multiple CPU cores while maintaining test isolation. Browser initialization is centralized in the setUp() method with a flexible parameter resolution strategy that prioritizes JVM arguments over TestNG parameters and configuration files.

3. Maven Configuration

3.1 Project Dependencies

The Maven configuration follows enterprise-grade dependency management with specific versions chosen for stability and compatibility. The pom.xml includes Selenium Java 4.18.1 for modern WebDriver capabilities, TestNG 7.9.0 for comprehensive test orchestration, and ExtentReports 5.1.1 for professional HTML reporting. Apache POI 5.2.3 and OpenCSV 5.8 enable robust data-driven testing capabilities, while WebDriverManager 5.7.0 handles cross-platform driver management. The Maven Surefire plugin 3.2.5 is configured to execute the TestNG suite with browser parameterization and parallel execution support.

3.2 Key pom.xml snippet

```
<dependencies>
  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.18.1</version>
  </dependency>
  <!-- WebDriverManager: auto downloads browser drivers -->
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>5.7.0</version>
  </dependency>
  <!-- TestNG -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.9.0</version>
  </dependency>
  <!-- ExtentReports -->
  <dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.1.1</version>
  </dependency>
  <!-- Apache POI for Excel -->
  <dependency>
    <groupId>org.apache.poi</groupId>
    <artifactId>poi-ooxml</artifactId>
    <version>5.2.3</version>
  </dependency>
  <!-- OpenCSV for CSV -->
  <dependency>
    <groupId>com.opencsv</groupId>
    <artifactId>opencsv</artifactId>
    <version>5.8</version>
```

```

    </dependency>
</dependencies>

```

4. Core Framework Components

4.1 BaseTest — The Foundation

BaseTest serves as the cornerstone of the framework, providing browser-agnostic WebDriver initialization and lifecycle management. The class implements a sophisticated browser selection strategy that prioritizes JVM arguments over TestNG parameters and configuration files, enabling flexible execution environments. It manages WebDriver instances through ThreadLocal storage for thread-safe parallel execution and integrates with TestListener via ITestContext for comprehensive reporting. The setUp() method handles browser-specific configurations including Chrome options for bot detection mitigation, Edge driver path resolution for network-constrained environments, and Firefox options for consistent maximized execution.

```

switch (browserName) {
    case "firefox":
        WebDriverManager.firefoxdriver().setup();
        FirefoxOptions ffOpts = new FirefoxOptions();
        ffOpts.addArguments("--start-maximized");
        webDriver = new FirefoxDriver(ffOpts);
        System.out.println("[BaseTest] Firefox launched");
        break;

    case "edge":
        // using local msedgedriver because azureedge.net is blocked on this
network
        EdgeOptions edgeOpts = new EdgeOptions();
        edgeOpts.addArguments("--start-maximized");
        edgeOpts.addArguments("--disable-notifications");
        edgeOpts.addArguments("--disable-blink-features=AutomationControlled");

        // resolve driver path relative to project root – works on any
machine
        String driverPath = System.getProperty("user.dir")
            + "/src/test/resources/drivers/msedgedriver.exe";
        java.io.File driverFile = new java.io.File(driverPath);

        if (driverFile.exists()) {
            // found in local driver – set it directly, skip network lookup
            System.setProperty("webdriver.edge.driver", driverPath);
            System.out.println("[BaseTest] edge driver loaded from project: "
+ driverPath);
        } else {

```

```

        // local driver missing – log clear instructions
        System.out.println("[BaseTest] msedgedriver.exe not found at: " +
driverPath);
        System.out.println("[BaseTest] download from:
https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/");
        System.out.println("[BaseTest] place it at:
src/test/resources/drivers/msedgedriver.exe");
        throw new RuntimeException(
            "msedgedriver.exe not found. Download Edge 145 driver from "
+
            "https://developer.microsoft.com/en-us/microsoft-
edge/tools/webdriver/ " +
            "and place at: src/test/resources/drivers/msedgedriver.exe"
        );
    }

    webDriver = new EdgeDriver(edgeOpts);
    System.out.println("[BaseTest] edge launched successfully");
    break;

default: // chrome
    WebDriverManager.chromedriver().setup();
    ChromeOptions chromeOpts = new ChromeOptions();
    chromeOpts.addArguments("--start-maximized");
    chromeOpts.addArguments("--disable-notifications");
    chromeOpts.addArguments("--disable-blink-
features=AutomationControlled");
    chromeOpts.addArguments("--no-sandbox");
    chromeOpts.addArguments("--disable-dev-shm-usage");
    chromeOpts.addArguments(
        "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        + "AppleWebKit/537.36 (KHTML, like Gecko) "
        + "Chrome/120.0.0.0 Safari/537.36");
    chromeOpts.setExperimentalOption("excludeSwitches",
        new String[]{"enable-automation"});
    chromeOpts.setExperimentalOption("useAutomationExtension", false);
    // Enable browser console logs (Selenium 4 feature)
    LoggingPreferences logPrefs = new LoggingPreferences();
    logPrefs.enable(LogType.BROWSER, Level.ALL);
    chromeOpts.setCapability("goog:loggingPrefs", logPrefs);
    webDriver = new ChromeDriver(chromeOpts);
    System.out.println("[BaseTest] Chrome launched");
    break;
}

```

4.2 ConfigReader — Externalized Configuration

ConfigReader implements a singleton pattern for centralized configuration management, loading properties from `src/test/resources/config.properties` during static initialization. This approach eliminates hardcoded values throughout the framework, enabling

environment-specific configurations without code changes. The class provides type-safe access to configuration values like `BASE_URL`, `BROWSER`, and user credentials, with built-in error handling for missing keys and comprehensive logging for troubleshooting configuration issues.

4.3 WaitUtil — Synchronization Strategy

WaitUtil encapsulates three distinct synchronization strategies to handle varying web application behaviors. The explicit wait methods (`waitForVisible`, `waitForClickable`) use `WebDriverWait` with `ExpectedConditions` for reliable element state verification, while the `fluentWait` method provides configurable polling intervals and exception ignoring for intermittent AJAX elements. The `setImplicitWait` method establishes baseline timing for `findElement` operations, though the framework primarily relies on explicit waits for precision.

```
/*
 * Fluent wait – polls every pollMs milliseconds for up to timeoutSec.
 * Ignores NoSuchElementException during polling.
 */
public WebElement fluentWait(By locator, int timeoutSec, int pollMs) {
    System.out.println("[WaitUtil] Fluent wait: " + locator
        + " | timeout=" + timeoutSec + "s | poll=" + pollMs + "ms");
    return new FluentWait<>(driver)
        .withTimeout(Duration.ofSeconds(timeoutSec))
        .pollingEvery(Duration.ofMillis(pollMs))
        .ignoring(NoSuchElementException.class)
        .until(d -> d.findElement(locator));
}
```

4.4 TestListener — Event-Driven Reporting

TestListener implements the `ITestListener` interface to provide comprehensive test execution monitoring and reporting through `ExtentReports`. The class initializes `ExtentReports` with HTML spark reporter, capturing system information and test metadata for rich reporting. It automatically captures screenshots on test failure using `ScreenshotUtil` and attaches them to the `ExtentReports`, creating a visual audit trail for debugging. The `onTestStart` method creates individual test nodes, while `onTestSuccess` and `onTestFailure` record appropriate outcomes with detailed logging.

```
@Override
public void onTestFailure(ITestResult result) {
    System.out.println("[TestListener] FAILED: " + result.getName());
    testNode.get().fail(result.getThrowable());

    // Get driver – try context first, then static accessor
    WebDriver driver = null;
    try {
        driver = (WebDriver) result.getTestContext().getAttribute("driver");
    } catch (Exception ignored) {}
}
```

```

        if (driver == null) driver = BaseTest.getDriver();

        if (driver != null) {
            String path = ScreenshotUtil.captureScreenshot(driver,
result.getName());
            if (!path.isEmpty()) {
                try {
                    testNode.get().addScreenCaptureFromPath(path, "Failure
Screenshot");
                    System.out.println("[TestListener] Screenshot attached: " +
path);
                } catch (Exception e) {
                    System.err.println("[TestListener] Screenshot attach error: "
+ e.getMessage());
                }
            }
        } else {
            System.err.println("[TestListener] Driver null – cannot capture
screenshot");
        }
    }
}

```

4.5 RetryAnalyzer — Handling Test Flakiness

RetryAnalyzer addresses the inherent instability of web applications by implementing automatic test retry logic for transient failures. The class tracks retry attempts with a configurable MAX_RETRY constant set to 2, allowing tests to recover from network delays, timing issues, or temporary server errors. By implementing the IRetryAnalyzer interface, it integrates seamlessly with TestNG's test lifecycle, providing clear logging of retry attempts and final failure reasons for comprehensive test analysis.

5. Test Suite Design

5.1 TestNG Suite Configuration

The TestNG suite is configured in testng.xml with parallel execution enabled at the class level and a thread count of 2 for optimal resource utilization. The suite is organized into five logical test groups covering different aspects of the application: registration, login, checkout, wait strategies, and Selenium 4 features. Each test group can execute independently or as part of the complete suite, with TestListener registered for comprehensive reporting and browser parameterization for cross-browser testing.

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="SAF-Suite"
    verbose="1"
    parallel="classes"
    thread-count="2">

```

```
<listeners>
  <listener class-name="com.saf.listeners.TestListener"/>
</listeners>

<!-- Default browser parameter for entire suite -->
<parameter name="browser" value="chrome"/>

<test name="SignUp-Tests" preserve-order="true">
  <parameter name="browser" value="chrome"/>
  <groups>
    <run>
      <include name="signup"/>
      <include name="smoke"/>
    </run>
  </groups>
  <classes>
    <class name="com.saf.tests.SignUpTest"/>
  </classes>
</test>

<test name="Login-Tests" preserve-order="true">
  <parameter name="browser" value="chrome"/>
  <groups>
    <run>
      <include name="login"/>
      <include name="smoke"/>
      <include name="datadriven"/>
    </run>
  </groups>
  <classes>
    <class name="com.saf.tests.LoginTest"/>
  </classes>
</test>

<test name="Checkout-Tests" preserve-order="true">
  <parameter name="browser" value="chrome"/>
  <classes>
    <class name="com.saf.tests.CheckoutTest"/>
  </classes>
</test>

<test name="Wait-Demo-Tests" preserve-order="true">
  <parameter name="browser" value="chrome"/>
  <classes>
    <class name="com.saf.tests.WaitDemoTest"/>
  </classes>
</test>

<test name="Selenium4-Tests" preserve-order="true">
```



```

    <parameter name="browser" value="chrome"/>
    <classes>
      <class name="com.saf.tests.Selenium4Test"/>
    </classes>
  </test>

  <test name="Locator-Demo-Tests" preserve-order="true">
    <parameter name="browser" value="chrome"/>
    <classes>
      <class name="com.saf.tests.LocatorDemoTest"/>
    </classes>
  </test>
</suite>

```

5.2 Test Classes Overview

Test Class	Methods	Data Rows	Total Runs	What it covers
SignUpTest	1	-	1	User registration flow with form validation
LoginTest	5	-	5	Valid/invalid login, hover/double-click, Excel-driven
LoginTest	testLoginCSVDri ven	3	3	CSV data-driven testing
CheckoutTest	1	-	1	Complete e-commerce checkout with address management
WaitDemoTest	3	-	3	Implicit, explicit, and fluent wait strategies
Selenium4Test	4	-	4	Relative locators, new tab/window, console logs
LocatorDemoTest	1	-	1	Absolute vs relative XPath comparison
Total	16	3	19	Comprehensive test coverage

6. Test Scenarios in Detail

6.1 Registration Flow (SignUpTest)

The `testRegisterNewUser` method demonstrates complete user registration automation using multiple locator strategies and interaction patterns. It generates unique email addresses using `System.currentTimeMillis()` to ensure data independence across test runs, preventing duplicate account errors. The test employs five distinct locator types: `By.linkText` for navigation, `By.id` for form fields, `By.cssSelector` for email input variation, and dynamic XPath with `contains()` for error message validation. `JavaScriptExecutor` is utilized for scrolling the register button into view before clicking, demonstrating advanced interaction techniques for elements outside the viewport.

6.2 Login Flow (LoginTest)

`LoginTest` implements comprehensive login validation through multiple test methods covering successful authentication, failure scenarios, and advanced user interactions. The `testValidLogin` method demonstrates standard login flow with logout verification, while `testInvalidLogin` showcases error message validation. The `testActionsHover` method uses `Actions.moveToElement()` to hover over the Computers menu and verify submenu appearance, while `testActionsDoubleClick` demonstrates `Actions.doubleClick()` on product titles with JavaScript scrolling for viewport positioning. Data-driven testing is implemented through both Excel and CSV DataProviders, with `testLoginExcelDriven` and `testLoginCSVDriven` methods iterating through multiple credential sets.

6.3 Checkout Flow (CheckoutTest)

The `testAddToCartAndCheckout` method orchestrates a complete e-commerce transaction from login through order confirmation. The test begins with user authentication, navigates to product catalog, adds items to cart using CSS selectors, and proceeds through a multi-step checkout process. It handles AJAX-loaded accordion sections by selecting saved addresses using `Select.selectByValue("4805620")` for both billing and shipping dropdowns, chooses Ground shipping method, selects Cash on Delivery payment option, and confirms order placement. Success verification uses CSS selector `.title strong` to locate the order completion message.

```
// Select saved address by value "4805620"
wait.waitForVisible(By.id("billing-address-select"), 15);
Select billingDrop = new Select(
    driver.findElement(By.id("billing-address-select")));
billingDrop.selectByValue("4805620");
System.out.println("[CheckoutTest] Billing: selected saved address - "
    + billingDrop.getFirstSelectedOption().getText());

// Click Continue (calls Billing.save() via JS)
WebElement billingNext = wait.waitForClickable(
    By.cssSelector("#billing-buttons-container input.button-1"), 10);
```

```
js.executeScript("arguments[0].click();", billingNext);
System.out.println("[CheckoutTest] Billing Continue clicked");
Thread.sleep(3000);

// — STEP 5: SHIPPING ADDRESS —————
System.out.println("[CheckoutTest] STEP 5: Shipping address");
wait.waitForVisible(By.id("shipping-address-select"), 20);
Select shippingDrop = new Select(
    driver.findElement(By.id("shipping-address-select")));
shippingDrop.selectByValue("4805620");
System.out.println("[CheckoutTest] Shipping: selected saved address – "
    + shippingDrop.getFirstSelectedOption().getText());
```

7. Locator Strategies Employed

7.1 Absolute vs Relative XPath Comparison (LocatorDemoTest)

The testAbsoluteVsRelativeXPath method demonstrates the critical difference between brittle absolute XPath expressions and robust relative XPath strategies. The test first attempts to locate elements using absolute XPath paths (/html/body/...) which fail with minor DOM changes, then showcases relative XPath approaches using contains(), starts-with(), and sibling relationships that maintain stability across page updates. This practical comparison highlights why relative locators are preferred for production test automation, showing how absolute paths break with structural changes while relative paths adapt to dynamic content.

7.2 Comprehensive Locator Strategies

Across the test suite, six distinct locator strategies were employed to demonstrate comprehensive element identification capabilities. The By.id strategy is used for form fields like FirstName, Email, and Password where stable identifiers are available. By.cssSelector provides flexible element targeting for product grids and dynamic elements, while By.xpath enables complex element relationships through absolute and relative paths. By.linkText is employed for navigation elements with stable text content, and By.name is used for search functionality. JavaScriptExecutor supplements these strategies for scrolling and attribute manipulation, particularly useful for elements outside the viewport or requiring property access.

The framework demonstrates both absolute XPath fragility and relative XPath robustness through LocatorDemoTest, showing how absolute paths break with minor DOM changes while relative paths maintain stability. Dynamic element handling is achieved through XPath contains() expressions and CSS attribute selectors, providing resilience against dynamically generated content or changing element states.

8. Wait Strategies

8.1 Implicit Wait

The `testImplicitWait` method demonstrates the baseline synchronization strategy by setting a global timeout on the `WebDriver` instance. This approach applies to all `findElement` calls automatically but shows limitations when dealing with AJAX content that loads after initial page load, as implicit waits cannot target specific element states or conditions.

8.2 Explicit Wait

The `testExplicitWait` method showcases precise element synchronization using `WebDriverWait` with `ExpectedConditions.visibilityOfElementLocated`. This strategy resolves timing issues in dynamic search functionality by waiting for specific element states rather than arbitrary time delays, providing reliable interaction with elements that appear after user actions or AJAX responses.

8.3 Fluent Wait

The `testFluentWait` method implements advanced synchronization with configurable polling intervals for elements that appear intermittently. Using a 300ms polling frequency over 15 seconds while ignoring `NoSuchElementException`, this approach optimizes test execution speed while maintaining reliability for elements like notification bars or dynamically loaded content that may not be immediately available.

9. Selenium 4 Features

The `Selenium4Test` class comprehensively demonstrates the latest Selenium 4 capabilities through practical implementation. Relative locators are showcased using all five position methods (`above`, `below`, `near`, `toLeftOf`, `toRightOf`) with a search box reference element, enabling element location based on visual positioning rather than DOM structure. New tab handling is implemented using `driver.switchTo().newWindow(WindowType.TAB)` to open and navigate to Google, demonstrating tab management capabilities. New window handling uses `WindowType.WINDOW` to open Wikipedia in a separate browser instance, with proper handle management for window switching. Browser console logs are captured through `LoggingPreferences` configuration and retrieved using `driver.manage().logs().get(LogType.BROWSER)`, providing valuable debugging information for JavaScript errors and network activity.

10. Data-Driven Testing

10.1 Excel-Based Testing (Apache POI)

ExcelUtil provides comprehensive Excel file handling using Apache POI library for reading test data and writing results back to spreadsheets. The readExcelData method processes the LoginData sheet, skipping headers and returning String arrays for test consumption. The writeResult method updates test outcomes in the last column of specified rows, enabling result tracking across multiple test executions. The createTestDataFileIfMissing method ensures test data availability by creating a sample Excel file with login credentials and a Result column for outcome tracking.

```
@DataProvider(name = "loginExcelData")
public Object[][] getExcelData() {
    try (FileInputStream fis = new
FileInputStream("src/test/resources/testdata.xlsx"));
        Workbook wb = new XSSFWorkbook(fis)) {

        Sheet sheet = wb.getSheet("LoginData");
        if (sheet == null) {
            System.err.println("[LoginTest] Sheet 'LoginData' not found");
            return new Object[0][0];
        }
        int rows = sheet.getLastRowNum();           // excludes header
        if (rows == 0) {
            System.err.println("[LoginTest] Sheet 'LoginData' has no data
rows");
            return new Object[0][0];
        }
        Row headerRow = sheet.getRow(0);
        if (headerRow == null) {
            System.err.println("[LoginTest] Sheet 'LoginData' has no header
row");
            return new Object[0][0];
        }
        int cols = headerRow.getLastCellNum();
        if (cols == 0) {
            System.err.println("[LoginTest] Sheet 'LoginData' has no
columns");
            return new Object[0][0];
        }
        DataFormatter fmt = new DataFormatter();
        String[][] data = new String[rows][cols];

        for (int r = 1; r <= rows; r++) {
            Row row = sheet.getRow(r);
            if (row == null) continue;
            for (int c = 0; c < cols; c++) {
                Cell cell = row.getCell(c);
```

```

        data[r - 1][c] = (cell == null) ? "" :
fmt.formatCellValue(cell);
    }
}
Object[][] raw = new Object[data.length][];
for (int i = 0; i < raw.length; i++) data[i] = raw[i];
System.out.println("[LoginTest] Excel DataProvider: " + raw.length +
" rows");
return data;
} catch (Exception e) {
    System.err.println("[LoginTest] Excel error, using fallback: " +
e.getMessage());
    return new Object[][]{{"skip@test.com", "skippass", "Fail"}};
}
}

```

10.2 CSV-Based Testing (OpenCSV)

CSV-based testing is implemented using OpenCSV library for lightweight data-driven test scenarios. The getCSVData method reads testdata.csv containing three invalid login credential sets, skipping the header row and returning Object arrays for TestNG DataProvider consumption. This approach provides a simple format for test data maintenance and enables rapid iteration through multiple test scenarios without the overhead of Excel file handling.

11. Reporting

The reporting infrastructure combines ExtentReports for rich HTML reporting with TestNG's native HTML output for comprehensive test execution documentation. ExtentReports is initialized in TestListener with system information capture including project details, author, and framework version. The HTML report provides an interactive dashboard with test categorization, status indicators, and screenshot attachments for failed tests. TestNG generates traditional HTML reports in the target/surefire-reports directory, offering a standardized view of test results with stack traces and execution timing. The dual reporting approach ensures compatibility with both CI/CD pipelines requiring HTML reports and manual test review processes.

12. Test Execution Results

Upon final execution of the complete test suite via mvn clean test, all 19 test cases passed successfully across Chrome, Firefox, and Edge browsers. The execution completed with zero failures, zero errors, and zero skipped tests, demonstrating framework stability and reliability. Total execution time averaged approximately 4+ minutes across all test classes, with parallel execution reducing overall duration. Tests were distributed across SignUpTest(1), LoginTest(8 including 3 CSV iterations), CheckoutTest(1),

WaitDemoTest(3), Selenium4Test(4), and LocatorDemoTest(1), providing comprehensive coverage of user interactions and framework features.

Test	Class	Status
testRegisterNewUser	SignUpTest	 PASS
testValidLogin	LoginTest	 PASS
testInvalidLogin	LoginTest	 PASS
testActionsHover	LoginTest	 PASS
testActionsDoubleClick	LoginTest	 PASS
testLoginExcelDriven	LoginTest	 PASS
testLoginCSVDriven (Row 1)	LoginTest	 PASS
testLoginCSVDriven (Row 2)	LoginTest	 PASS
testLoginCSVDriven (Row 3)	LoginTest	 PASS
testAddToCartAndCheckout	CheckoutTest	 PASS
testImplicitWait	WaitDemoTest	 PASS
testExplicitWait	WaitDemoTest	 PASS
testFluentWait	WaitDemoTest	 PASS
testRelativeLocators	Selenium4Test	 PASS
testNewTabHandling	Selenium4Test	 PASS
testNewWindowHandling	Selenium4Test	 PASS
testBrowserConsoleLogs	Selenium4Test	 PASS
testAbsoluteVsRelativeXPath	LocatorDemoTest	 PASS

13. Challenges and Solutions

The project encountered and resolved several significant challenges during development. The original plan to use demo.opencart.com was blocked by Cloudflare bot protection, requiring a strategic pivot to demowebshop.tricentis.com, which ultimately provided a more stable testing environment. Registration button identification presented a challenge with the HTML using an INPUT tag instead of BUTTON, requiring CSS selector adaptation for reliable element targeting. The checkout process revealed complex AJAX accordion behavior that necessitated saved address selection using selectByValue("4805620") to bypass dynamic form loading. These solutions demonstrate practical problem-solving abilities and adaptation to real-world web application complexities.

14. Conclusion

This Selenium Automation Framework successfully demonstrates comprehensive mastery of WebDriver automation concepts through practical implementation of a production-ready testing solution. The project showcases advanced Selenium 4 features, robust synchronization strategies, and enterprise-level reporting capabilities while maintaining clean code architecture and comprehensive test coverage. The framework provides a solid foundation for extension with Page Object Model implementation, CI/CD pipeline integration, and advanced reporting tools like Allure Reports. Through systematic implementation of best practices including parallel execution, data-driven testing, and cross-browser compatibility, this project establishes a professional standard for web application automation that can scale to enterprise testing requirements.

Author: Sahil Singh